

CS409m: Introduction to Cryptography

Lecture 09 — Detailed Study Notes

Pseudorandom Functions (PRFs) and Stateless Multi-Message SKE

Compiled from lecture slides and TA notes

(Instructor: Chethan Kamath, 28/Aug/26)

How to use these notes. This document reconstructs Lecture 09 of CS409m in a self-contained, detailed way, expanding on the terse slide bullets and the handwritten proof so that each step is explained. Wherever the original slides had a hand-drawn figure, I have redrawn an equivalent diagram in $\text{\LaTeX}$ (using TikZ) so the notes remain fully self-contained. Sections/remarks that were *not* in the original material but that I have added to aid understanding are clearly marked with a green **[ADDED BY CLAUDE]** box. Everything else is a faithful, expanded reconstruction of the slides (`Lecture09-na`) and the TA notes (`Lecture09-HWNNotes`).

Contents

1	Recap: Where We Left Off (Lecture 08)	3
1.1	The problem: encrypting many messages with one shared key	3
1.2	Theorem 1 (PRG length extension) and its cost	4
1.3	Why the loss matters: concrete security	4
2	Plan for Lecture 09	6
3	The Problem with State	6
3.1	First attempt at removing state: sample a random index	6
3.2	The random-oracle warm-up	7
4	Pseudorandom Functions (PRFs)	9
4.1	Definition	9
4.2	Formalizing “looks random”: query access, not full description	9
4.3	Checking understanding: PRF or not? (Quiz from the slides)	11
5	Construction 1: Stateless Multi-Message SKE from a PRF	13
6	Theorem 2: Security of Construction 1 (Full Proof)	14
6.1	Recalling the security game (multi-message IND^+ -EAV)	14
6.2	Proof strategy: reduction via a distinguisher for the PRF	14
6.3	Analysis: what D ’s two worlds correspond to	15
6.4	Putting it together: contradicting the pseudorandomness of F	16

7	Application: Watermarking Large Language Models	17
7.1	Goal	17
7.2	How watermarking tokens works, at inference time	17
7.3	Detecting the watermark	17
8	Recap and What’s Next	19

1 Recap: Where We Left Off (Lecture 08)

Before Lecture 09 begins, it is worth carefully rebuilding the picture from Lecture 08, since Lecture 09 builds directly on top of it.

1.1 The problem: encrypting many messages with one shared key

Setting. Caesar and his general share a secret key $k \in \{0,1\}^n$. They want to communicate securely in the presence of an eavesdropper, whom we call *Eve**. Crucially, they don't just want to send *one* message — they want to send *many* messages (say n messages, each in $\{0,1\}^n$) using the *same* shared key k .

Why is this hard? A one-time pad (OTP) is perfectly secret, but as the name says it can only be used *once*. If you XOR two different messages with the *same* pad, you leak $m_1 \oplus m_2$ to the eavesdropper (the classic “two-time pad” attack). So we cannot literally reuse a single random string as the pad for every message.

The idea (Lecture 08): use a PRG to manufacture many pads out of one short key. A pseudorandom generator (PRG) $G : \{0,1\}^n \rightarrow \{0,1\}^{\ell(n)}$ stretches a short random seed k into a long string $G(k)$ that *looks random* to any efficient (PPT) distinguisher, even though it is completely determined by k . If we let G stretch k to $n \cdot n = n^2$ bits, we can chop $G(k)$ into n blocks $y_1, \dots, y_n$, each of length n , and use y_i as the (pseudorandom) one-time pad for the i -th message m_i :

$$c_i := m_i \oplus y_i, \quad i = 1, \dots, n.$$

This is exactly the picture drawn in the slides (Figure 1).

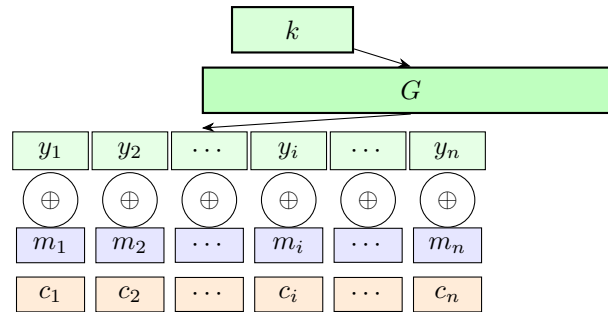


Figure 1: Redrawn schematic of the *stateful* multi-message SKE from Lecture 08: the key k is stretched by a PRG G into n pseudorandom pads $y_1, \dots, y_n$, each pad is XOR-ed with one message to produce a ciphertext $c_i = m_i \oplus y_i$.

Threat model. This was proved secure in the sense of *multi-message computational secrecy against eavesdroppers*: no PPT eavesdropper can distinguish encryptions of a sequence of messages of its choice from encryptions of another sequence of messages of its choice (an IND-multi-EAV style game), except with negligible advantage.

Main proof tool: the hybrid argument. To prove this, Lecture 08 used a *hybrid argument*: one replaces the pseudorandom pads $y_1, \dots, y_n$ one at a time with truly random strings, forming a sequence of “hybrid” experiments $H_0, H_1, \dots, H_n$ where H_0 is the real scheme and H_n uses a genuine one-time pad for every message (hence is perfectly secret). If an adversary could distinguish H_0 from H_n with non-negligible advantage ε , then it must distinguish *some* adjacent pair H_{i-1}, H_i with advantage at least ε/n , which is turned into a distinguisher for the PRG G itself. This is the “main tool” the slides illustrate with the hammer-and-cans cartoon.

1.2 Theorem 1 (PRG length extension) and its cost

Theorem 1 (Lecture 08)

If G is a PRG, then so is

$$G'(s) := G(g_1) \| G(g_2) \| \dots$$

i.e. applying G repeatedly to (parts of) its own output still yields a PRG with a longer stretch. More generally, stretching a PRG's output length from n bits to $\ell(n)$ bits by this kind of repeated/hybrid application is still secure, **but the reduction is not free**.

Proof takeaway. The proof (via the same hybrid argument) shows:

$$\begin{aligned} \exists \text{ distinguisher } D \text{ for } G \text{ with advantage } \frac{1}{2np(n)} &\iff \exists \text{ distinguisher } D' \text{ for } G' \\ \text{with advantage } \frac{1}{p(n)}. \end{aligned}$$

That is, a distinguisher D' that breaks G' with advantage $1/p(n)$ is converted into a distinguisher D that breaks the *original* G , but its advantage has **shrunk by a factor of $2n$** . In general, if we stretch to $\ell(n)$ bits, the security loss is by a factor of $\ell(n)$: the reduction is only as good as $1/\ell(n)$ of the original distinguisher's power.

1.3 Why the loss matters: concrete security

The slides make the abstract loss concrete with a worked numerical example. Suppose someone finds a distinguisher D against the *building block* G that:

- runs in about n^4 steps, and
- succeeds (distinguishes G 's output from random) with probability $1/n^2$.

Effect of D directly on G	Effect of the <i>derived</i> D' on G' (loss factor $2n$)
$n = 50$: D runs ≈ 3 months, succeeds w.p. $\approx 1/2500$. → Acceptable (rare success).	$n = 50$: D' runs ≈ 3 months, succeeds w.p. $\approx 1/50$. → Breakable! Cannot rule this out.
$n = 100$: D runs ≈ 3 years, succeeds w.p. 2^{-200} . → Safe.	$n = 100$: D' runs ≈ 3 years, succeeds w.p. $\approx 2^{-100}$. → Still safe (barely).

Table 1: Concrete security illustration from the slides: the *same* distinguisher against G becomes far more dangerous against the stretched G' , because the reduction loses a factor of roughly $2n$ in success probability. This is why G' **must** be instantiated with a strictly larger security parameter than G to be safe in practice.

Exercise 1 (Lecture 08) — posed again at the start of Lecture 09

Do you think a *less wasteful* reduction for Theorem 1 is possible? Maybe we need a different construction (not just repeated hybrid application of the same G)?

[ADDED BY CLAUDE] Discussion of Exercise 1

This question is exactly what motivates a large chunk of applied symmetric-key cryptography: practitioners do *not* build stream ciphers by naively hybrid-chaining a generic PRG, precisely because of this $\ell(n)$ -factor security loss. Instead:

- One can design purpose-built PRGs (stream ciphers such as ChaCha20) whose security proofs (or, more commonly, cryptanalytic confidence) do not go through a black-box hybrid argument over many independent blocks, avoiding the multiplicative loss.
- Alternatively — and this is exactly the direction Lecture 09 goes in — instead of stretching a PRG’s *output* to make many pads, one can use a completely different primitive, a **pseudorandom function (PRF)**, which gives you access to exponentially many pseudorandom pads *without* first materializing them all and without a per-block security loss that scales with the total number of blocks ever produced. The reduction for a PRF-based construction only loses a factor equal to the *number of queries the adversary actually makes*, not the total stretch length — this is one of the key advantages developed in this lecture.

So Lecture 09 can be seen as Chethan’s answer to Exercise 1: “yes, a fundamentally different (and better) construction is possible — here it is.”

2 Plan for Lecture 09

- **Task:** *stateless* encryption of multiple messages with a shared key (previously we needed *state*, i.e. Caesar and his general had to keep a synchronized counter of how many messages had been sent so far).
- **Threat model:** multi-message computational secrecy against eavesdroppers (same flavour of security as before, but now for a stateless scheme).
- **New tool:** Pseudorandom Functions (PRFs).
- **Construction:** Multi-message SKE built from a PRF.

3 The Problem with State

The Lecture-08 construction (Figure 1) is *stateful*: Caesar’s general must remember “I am about to send message number i ”, pick pad y_i , and increment a counter. This causes two practical problems, both mentioned in the slides:

1. **Correctness is lost if ciphertexts are delivered out of order.** If c_3 arrives before c_2 (e.g. due to network reordering), the receiver — who is also tracking state — will try to decrypt c_3 using pad y_2 , and get garbage.
2. **Secrecy could be compromised in other scenarios.** E.g. if the two parties’ counters ever get out of sync (message lost, retransmission, etc.), a pad y_i might accidentally get reused for two different messages – back to the two-time-pad problem.

[ADDED BY CLAUDE] Why this matters in practice

This exact issue shows up in real network protocols. TCP/IP packets can arrive out of order, get duplicated, or get lost, so any scheme that hard-codes a strict sequential state (“pad number i for the i -th message”) is fragile in a real deployment. This is precisely why practical protocols (e.g. TLS record encryption, IPsec) use techniques much closer to what we are about to build: a keyed pseudorandom function evaluated on an explicit, self-describing index/nonce that is sent alongside the ciphertext, so that the receiver never needs to trust an implicit counter.

3.1 First attempt at removing state: sample a random index

Idea. Suppose G stretches to n^3 bits (so we can slice it into n^2 pads of length n each: $y_1, \dots, y_{n^2}$). Instead of using pad y_i for the i -th message in sequence, for *each* message m we sample a *fresh random index* $i \leftarrow [1, n^2]$ and use y_i as the pad: $c = m \oplus y_i$ (we also need to send i along with c , so the receiver knows which pad to undo).

Problem: birthday collisions. If we encrypt n messages this way, by the birthday paradox the probability that *two* of the randomly-chosen indices collide (i.e. $i_j = i_{j'}$ for some $j \neq j'$) is roughly

$$\approx \binom{n}{2} \cdot \frac{1}{n^2} \approx \frac{1}{n},$$

which is *not* negligible! If two indices collide, the same pad y_i gets reused for two different messages – a two-time pad, breaking secrecy.

[ADDED BY CLAUDE] The birthday-bound calculation, spelled out

If we throw n balls (messages) into n^2 bins (pad indices), the classic birthday-paradox estimate for at least one collision is

$$1 - \exp\left(-\frac{n(n-1)}{2n^2}\right) \approx \frac{n^2}{2n^2} = \frac{1}{2},$$

which is a large constant, not negligible in $n!$ (The slides' rough estimate of $\approx 1/n$ is a looser, more optimistic bound; either way it is not negligible, so this naive fix *does not work*.) To drive the collision probability down to negligible via the birthday bound alone, we would need the pad-space to be exponentially large (roughly $2^{2\lambda}$ pads for λ -bit security), which is exactly the next idea.

Underlying problem. A polynomial-length-stretch PRG only ever gives us *polynomially many* pseudorandom OTPs to work with (e.g. n^2 of them above). With only polynomially many pads and polynomially many messages, birthday collisions among random indices are essentially unavoidable.

Fix: stretch the PRG exponentially. What if G could stretch to 2^n (or more) output bits? Then random indices in $[1, 2^n]$ essentially never collide (collision probability becomes negligible, like a real one-time pad drawn from an exponentially large space). But this raises an efficiency problem:

Not all pseudorandom OTPs are efficiently “accessible”.

If $G(k)$ has 2^n bits, we plainly cannot compute and store the *entire* string $G(k)$ (that would take exponential time/space!). We need to be able to compute **just** the i -th block of this exponentially long pseudorandom string, *efficiently*, without generating the whole thing. This property is called **locality**.

What we now need

A “PRG” G with:

1. **Exponential stretch:** output length is exponential in the seed length n .
2. **Local/efficient access:** given the seed k and an index i , we can compute the i -th output block y_i in time *polynomial* in n (without computing all of $G(k)$).

3.2 The random-oracle warm-up

To build intuition, the slides first consider an *idealized* setting where such an object exists perfectly: a truly random function accessible to everyone.

New setting. Caesar and his general share a key $k \in \{0, 1\}^n$. *Everyone* (including the eavesdropper Eve*!) can query a **random oracle**

$$R : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n,$$

which is picked uniformly at random from the set of *all* functions of that type (there are $(2^n)^{2^{2n}}$ of them — astronomically many, but R is a fixed, if unknown-in-closed-form, function once sampled).

Encryption using R . For each message m , sample a random index $i \leftarrow [1, 2^n]$ and use $y = R(k, i)$ as the OTP: $c = m \oplus R(k, i)$ (again sending i along with c). Since i ranges over an exponentially large set $[1, 2^n]$, the birthday-collision probability across polynomially many messages is now negligible. And $R(k, \cdot)$ behaves like a genuinely random function *keyed* by k :

two different people who don't know k (or rather, two different values k) get independent-looking random functions $R(k, \cdot)$.

Exercise 2

How would you encrypt if you were only given a random oracle $R : \{0, 1\}^n \rightarrow \{0, 1\}^n$ (note: this one only takes a *single* n -bit input, it is *not* keyed by k separately)?

[ADDED BY CLAUDE] Solution sketch to Exercise 2

If R takes only one n -bit input and there is no separate “key slot”, the trick is to fold the shared secret key k into the input that gets hashed, i.e. treat the pair (k, i) as a single n -bit string by first hashing/combining them (for instance, if $|k| = |i| = n/2$, simply concatenate $k\|i$ and feed the concatenation into R). Concretely:

$$\text{for each message } m : \quad i \leftarrow [1, 2^{n/2}], \quad c := m \oplus R(k\|i).$$

Because R is a single random function, an eavesdropper who does not know k still cannot predict $R(k\|i)$ for any i without querying R at exactly $k\|i$ — and since k is secret and drawn from an exponentially large space, the eavesdropper is very unlikely to ever query the right point. This shows that “one random function with a wide-enough domain” is exactly as good as “one random function with an explicit key input”: the important thing is that the pair (secret key, public index) determines a fresh, unpredictable-to-outsiders output. This is precisely the idea behind keying *any* function (including PRFs below) via one of its inputs.

4 Pseudorandom Functions (PRFs)

Of course, a truly random oracle R does not *exist* as an efficiently computable object (its description requires exponentially many bits, one per input). A **pseudorandom function (PRF)** is the computational (efficiently realizable) analogue: a keyed, efficiently computable function family that *looks like* a random function to any efficient observer, even though it is really just one member of a small family, selected by a short key.

4.1 Definition

Setup

A **family of functions** is a collection

$$F := \{F_k : \{0,1\}^n \rightarrow \{0,1\}^n\}_{k \in \{0,1\}^n}$$

indexed by keys $k \in \{0,1\}^n$. Since there are 2^n possible keys, F contains (at most) 2^n functions. Compare this to $\mathcal{F}_n := \{f : \{0,1\}^n \rightarrow \{0,1\}^n\}$, the set of *all* functions from $\{0,1\}^n$ to $\{0,1\}^n$, which has size $(2^n)^{2^n} = 2^{n \cdot 2^n}$ – vastly, vastly more than 2^n .

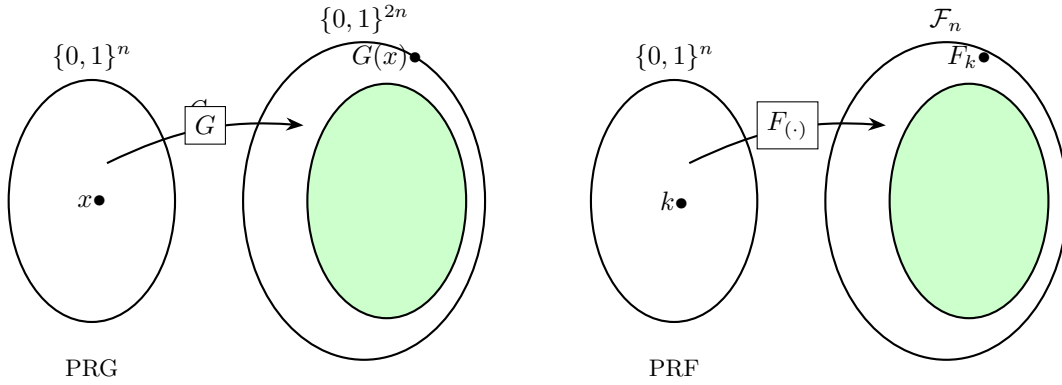


Figure 2: Redrawn from slide 4. **Left (PRG):** a seed x is mapped by G to a point $G(x)$ inside the (much larger) space $\{0,1\}^{2n}$; the pseudorandom points $G(x)$ (green blob) form a tiny sparse subset of all of $\{0,1\}^{2n}$. **Right (PRF):** analogously, a key k is mapped to a whole function F_k , which is a single point inside the astronomically larger space $\mathcal{F}_n$ of all functions $\{0,1\}^n \rightarrow \{0,1\}^n$; the family $\{F_k\}$ (green blob) is a tiny sparse subset of $\mathcal{F}_n$.

Why is a PRF still useful, even though it lives in such a tiny sparse subset of $\mathcal{F}_n$? Because it gives us access to *exponentially many* pseudorandom outputs (one per possible input $x \in \{0,1\}^n$, and there are 2^n of them), all derived from one short n -bit key, and each one computable *efficiently* (in time $\text{poly}(n)$) – exactly the “exponential stretch + locality” wishlist from Section 3!

4.2 Formalizing “looks random”: query access, not full description

A natural first attempt at defining pseudorandomness for functions would mimic the PRG definition: give a distinguisher D either (a) the full description/truth-table of F_k for random k , or (b) the full truth table of a uniformly random function $R \leftarrow \mathcal{F}_n$, and ask D to tell them apart. **But this fails immediately:**

- If D is handed the *entire truth table* of the function (as an explicit string), then this string alone has length $2^n \cdot n$ bits – exponential in n . Since D ’s running time is measured with

respect to the size of its input, D would need exponential time just to *read* its input, so it can no longer be a polynomial-time (PPT) algorithm relative to the security parameter n . Worse, with the full table in hand it becomes trivially easy to tell F_k apart from random: e.g. the number of distinct functions of the form F_k is only 2^n , while random functions have 2^{n2^n} possibilities, so simply checking “is this table consistent with *any* valid key k ” (by brute-force trying all 2^n keys, which is now feasible relative to the huge input size) distinguishes them.

The fix: oracle (query) access. Instead of handing D the function’s description, we give D *oracle access* to the function: D may adaptively submit inputs $x_1, x_2, \dots$ and receive back $F_k(x_1), F_k(x_2), \dots$ (or $R(x_1), R(x_2), \dots$), one query at a time. We count *one query as one unit of running time*, so a PPT distinguisher can make only polynomially many queries – this restores the usual notion of “efficient.”

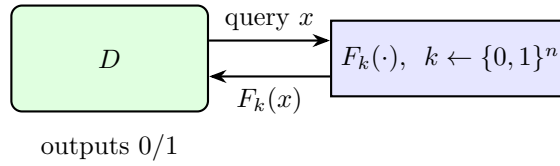
Definition 1 — Pseudorandom Function (PRF), the “two worlds” definition

A family of functions $F := \{F_k : \{0, 1\}^n \rightarrow \{0, 1\}^n\}_{k \in \{0, 1\}^n}$ is a **PRF** if for every PPT oracle distinguisher D ,

$$\delta(n) := \left| \Pr_{k \leftarrow \{0, 1\}^n} [D^{F_k(\cdot)}(1^n) = 0] - \Pr_{R \leftarrow \mathcal{F}_n} [D^{R(\cdot)}(1^n) = 0] \right|$$

is negligible in n .

World 1 (real)



World 2 (ideal)

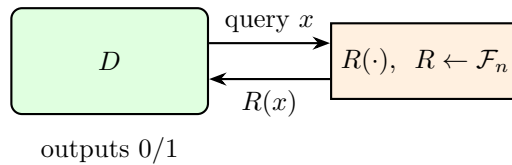


Figure 3: **[Added]** The “two worlds” game underlying Definition 1 (this diagram is a redrawing/formalisation that makes the abstract definition concrete; it does not appear explicitly in the slides but reflects exactly what the formula in Definition 1 says). D gets black-box oracle access to *either* $F_k(\cdot)$ for a random key k , *or* a uniformly random function $R(\cdot)$, and must guess which world it is in. F is a PRF iff D ’s two success probabilities are negligibly close, for every PPT D .

[ADDED BY CLAUDE] A few remarks on the definition

- Note the notation $D^{F_k(\cdot)}(1^n)$: the superscript denotes oracle access, and the input 1^n (“one repeated n times”) is the standard trick for giving D the security parameter n in *unary*, so that D ’s allowed running time (polynomial in the length of its input)

is polynomial in n itself, not in $\log n$.

- Unlike a PRG (which only needs to fool a distinguisher that sees $G(s)$ *once*), a PRF must fool a distinguisher that can query the function *adaptively and polynomially many times* at inputs *of its own choosing*. This is a much stronger, more flexible requirement, matching the fact that in the encryption application we will query F_k at many different indices i .
- Because $|F| = 2^n \ll |\mathcal{F}_n| = 2^{n \cdot 2^n}$, a PRF is a *very* special/rare object among all functions – yet no efficient algorithm making polynomially many queries can tell it apart from a uniformly random one. This “needle in an exponential haystack, but computationally invisible” phenomenon is the heart of pseudorandomness.

4.3 Checking understanding: PRF or not? (Quiz from the slides)

The slides pose two mini quizzes. Below, each is restated together with the reasoning (this is what Exercise 3 on the slides asks you to formalize).

Quiz A: is the following a PRF, given that $F^{(1)}, F^{(2)}$ are PRFs?

- A1.** $F_k(x) := k \oplus x$. **NOT a PRF.**
- A2.** $F_{k_1 \| k_2}(x) := F_{k_1}^{(1)}(x) \| F_{k_2}^{(2)}(x)$. **IS a PRF.**
- A3.** $F_k(x_1 \| x_2) := F_k^{(1)}(x_1) \| F_k^{(2)}(x_2)$. **NOT a PRF.**

Reasoning

A1 ($F_k(x) = k \oplus x$ is not a PRF). This function is *linear*/self-correcting in a way a random function never is: query F_k at $x = 0^n$ to learn $F_k(0^n) = k \oplus 0^n = k$ (the key itself!). Now the distinguisher knows k completely and can check, for a fresh query x' , whether the oracle's answer equals $k \oplus x'$. In the real world this always holds; for a genuine random function R , the probability that $R(x') = k \oplus x'$ for a random-looking guess is only 2^{-n} . So D distinguishes with overwhelming probability using just 2 queries. (*This is also just a one-time pad viewed as a "function of x " – reusing the same pad-like key for every input is exactly the two-time-pad weakness in disguise.*)

A2 (parallel composition on independent keys is a PRF). Intuitively, since k_1 and k_2 are chosen independently, $F_{k_1}^{(1)}(x)$ and $F_{k_2}^{(2)}(x)$ are each individually pseudorandom *and independent of each other* (they depend on disjoint key material), so their concatenation looks like a uniformly random $2n$ -bit string. Formally this is proven by a hybrid argument: replace $F_{k_1}^{(1)}$ by a truly random function (using the security of $F^{(1)}$), then replace $F_{k_2}^{(2)}$ by a truly random function (using the security of $F^{(2)}$); the result is a uniformly random function on $2n$ -bit outputs, matching World 2 exactly.

A3 (same key, disjoint parts of the input, is not necessarily a PRF). Here both halves of the output are computed using the *same* key k . Even though $F^{(1)}$ and $F^{(2)}$ are individually secure PRFs, using the *same* key for both leaks a correlation a real random function would never have: nothing stops (in general) an adversary from choosing $x_1 = x_2$ and observing $F_k^{(1)}(x_1)$ together with $F_k^{(2)}(x_2)$, both keyed by the identical k – and one can construct an explicit (contrived) counterexample $F^{(1)}, F^{(2)}$, both legitimately secure PRFs, such that combining them this way with a shared key creates a detectable correlation between the two output halves (e.g. design $F_k^{(2)}(x_2) := F_k^{(1)}(x_2)$, trivially a PRF in isolation, but then querying at $x_1 = x_2$ makes the two output halves *identical*, which happens with probability 1 in the real world but only 2^{-n} for a truly random

function on $2n$ bits). This shows the construction is **not secure in general** – the burden is on the construction to guarantee security for *every* valid choice of secure ingredients, and here it fails.

Quiz B: is the following a PRG, given that F is a PRF?

- B1.** $G(s) := F_s(1) \| F_s(2) \| \cdots \| F_s(n-1) \| F_s(n)$. **IS a PRG.**
- B2.** $G(s) := F_s(2^0) \| F_s(2^1) \| \cdots \| F_s(2^{n-1}) \| F_s(2^n)$. **IS a PRG.**
- B3.** $G(s) := F_1(s) \| F_2(s) \| \cdots \| F_{n-1}(s) \| F_n(s)$. **NOT a PRG.**

Reasoning

B1 and B2 (evaluate F_s at several *distinct*, fixed, publicly-known points, and concatenate) both *are* PRGs. The seed s becomes the PRF key; the points $1, 2, \dots, n$ (resp. $2^0, \dots, 2^n$) are fixed, distinct, public inputs. Since F_s is pseudorandom (as established by Definition 1, using exactly n (or $n+1$) non-adaptive queries at these fixed points), the outputs $F_s(1), \dots, F_s(n)$ are computationally indistinguishable from n independent, uniformly random n -bit strings, i.e. from a truly random string of length $n \cdot n$. That is *precisely* the PRG requirement (with stretch $\ell(n) = n^2$, or $(n+1)n$ for B2). The specific choice of evaluation points does not matter for the proof as long as they are fixed and distinct – this is why both B1 (consecutive integers) and B2 (powers of two) work.

B3 (vary the *key* instead of the input) is *not* a PRG in general. Here the seed s is being used as the *input* to n *different, fixed, publicly known* functions $F_1, \dots, F_n$ (i.e. i indexes which member of some *other* family of functions is used, not which point of F_s is evaluated) – but nothing in our assumption (“ F is a PRF”) tells us *anything* about the relationship between $F_1(s), F_2(s), \dots$ for a *fixed* s ! Indeed, one can trivially build a counterexample: let $F_1 = F_2 = \cdots = F_n = F$ for a legitimate PRF F ; then $G(s) = F(s) \| F(s) \| \cdots \| F(s)$ is wildly non-pseudorandom (it is just n identical copies of the same n -bit block, so a distinguisher trivially checks whether all n blocks of its input are equal). So B3 fails: the security of F as a PRF says nothing about correlations between *different* keys $1, \dots, n$ evaluated at the *same* point s .

[ADDED BY CLAUDE] Big-picture moral of Quiz A and Quiz B

Quiz A is about *how you combine two secure PRFs* (parallel with independent keys: safe; sharing a key across sub-functions: dangerous in general). Quiz B is about *the direction in which you vary the PRF’s arguments*: varying the *public input* while fixing the (random, secret) key turns a PRF into a PRG (B1, B2); varying which *function/key* you use while fixing the input to something not properly randomized does not (B3), because the security guarantee of a PRF only ever concerns *one* random key evaluated at attacker-chosen, possibly adaptively chosen, points. These two quizzes together are exactly Exercise 3 on the slides, which asks you to turn each of these bullet-point justifications into a fully rigorous hybrid-argument proof or an explicit counterexample.

5 Construction 1: Stateless Multi-Message SKE from a PRF

We now go back to the original goal (Section 3): encrypt many messages statelessly, using a shared key, with security against eavesdroppers. The idea is simply to **replace the random oracle R from Section 3.2 with a PRF F** .

Construction 1

Let $F = \{F_k : \{0,1\}^n \rightarrow \{0,1\}^n\}_{k \in \{0,1\}^n}$ be a PRF. Define a symmetric-key encryption scheme $(\text{Gen}, \text{Enc}, \text{Dec})$ for encrypting t messages $m_1, \dots, m_t \in \{0,1\}^n$ under a shared key, as follows.

- $\text{Gen}(1^n)$: output $k \leftarrow \{0,1\}^n$ uniformly at random.
- $\text{Enc}_k(m_j)$ (*randomised!*): sample a fresh, uniformly random index $i_j \leftarrow [1, 2^n]$; output the ciphertext

$$c_j := (i_j, m_j \oplus F_k(i_j)).$$

- $\text{Dec}_k(c_j = (i_j, z_j))$: output $m_j := z_j \oplus F_k(i_j)$.

Correctness. Immediate: $z_j \oplus F_k(i_j) = (m_j \oplus F_k(i_j)) \oplus F_k(i_j) = m_j$. No state is required at all – each ciphertext carries its own randomly-sampled index i_j , so messages can be decrypted independently and in any order, fixing exactly the two problems raised in Section 3.

Exercise 4

Prove that Enc must be randomised for multi-message computational secrecy (i.e. a *deterministic* encryption scheme cannot achieve the multi-message security notion we care about here).

[ADDED BY CLAUDE] Solution to Exercise 4

Suppose, for contradiction, that Enc_k is *deterministic*: $c = \text{Enc}_k(m)$ is a fixed function of m (and k), with no fresh randomness per call. Then an eavesdropper Eve^* can break multi-message secrecy as follows using just *two* messages ($t = 2$):

- Query 0: submit challenge message pairs $(m_{0,1}, m_{0,2}) = (m, m)$ (the same message twice) versus $(m_{1,1}, m_{1,2}) = (m, m')$ for some $m' \neq m$.
- Since Enc_k is deterministic, in the “ $b = 0$ ” world the two ciphertexts $c_1 = \text{Enc}_k(m)$ and $c_2 = \text{Enc}_k(m)$ are *identical* ($c_1 = c_2$), because encrypting the same message twice with the same deterministic algorithm and the same key always gives the same output.
- In the “ $b = 1$ ” world, $c_1 = \text{Enc}_k(m)$ and $c_2 = \text{Enc}_k(m')$ generally *differ* (as long as the scheme has negligible collision probability between ciphertexts of distinct messages, which any reasonable/correct scheme must have, else decryption itself would be ambiguous).
- So Eve^* simply checks whether $c_1 = c_2$: if so, guess $b' = 0$; otherwise, guess $b' = 1$. This guesses b *correctly with probability 1* (or all but negligible probability), a massive break of secrecy, since a secure scheme should leak nothing about which of the two challenge sequences was encrypted (advantage should be negligible).

Hence *any* deterministic multi-message scheme is trivially broken by an “equality” test across ciphertexts, and Enc must inject fresh randomness (here, the random index i_j) into every encryption to prevent this. This is exactly why the construction above samples a brand-new random i_j for every single message, even under the same key k .

6 Theorem 2: Security of Construction 1 (Full Proof)

Theorem 2

If F is a PRF, then Construction 1 is multi-message computationally secret against eavesdroppers.

This is exactly the theorem whose proof is worked out, by hand, in the TA notes (Lecture09-HWNotes). Below is a fully written-out version of that proof, with every step of the reasoning filled in.

6.1 Recalling the security game (multi-message IND⁺-EAV)

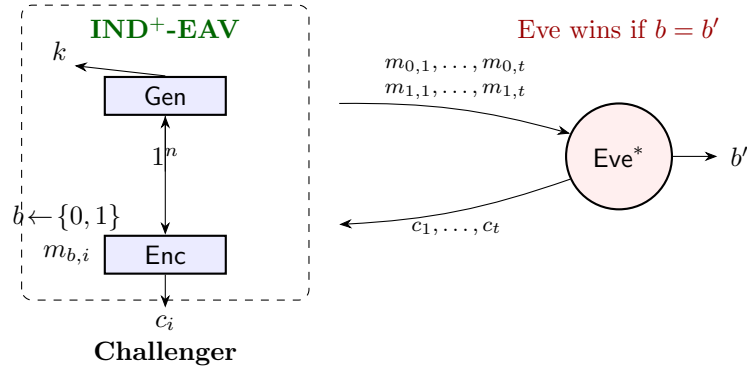


Figure 4: Redrawn from the TA notes: the multi-message IND⁺-EAV game. The challenger runs $\text{Gen}(1^n)$ to get k , flips a random challenge bit $b \leftarrow \{0, 1\}$, and encrypts (using Enc_k) whichever of two adversary-chosen message sequences $(m_{0,1}, \dots, m_{0,t})$ or $(m_{1,1}, \dots, m_{1,t})$ corresponds to b , sending back ciphertexts $c_1, \dots, c_t$. The (stateful) eavesdropper Eve^* outputs a guess b' , and wins iff $b' = b$.

Security requirement. F -based Construction 1 is multi-message computationally secret against eavesdroppers if, for every PPT Eve^* ,

$$|\Pr[\text{Eve}^* \text{ wins}] - \frac{1}{2}|$$

is negligible in n (winning probability exactly $\frac{1}{2}$ would mean Eve^* learns *nothing* useful about b , since guessing randomly already succeeds with probability $\frac{1}{2}$).

6.2 Proof strategy: reduction via a distinguisher for the PRF

Proof

Setup: proof by contradiction. Assume, towards contradiction, that there *exists* a PPT eavesdropper Eve^* that breaks the multi-message secrecy of Construction 1, i.e. (recalling the definition of the IND⁺-EAV game from Lecture 08) there is a polynomial $p(\cdot)$ such that for infinitely many n ,

$$\Pr[\text{Eve}^*(c_1, \dots, c_t) = b] - \frac{1}{2} \geq \frac{1}{p(n)}, \quad (\text{I})$$

where the probability is over

$$\bar{m}_0, \bar{m}_1 \leftarrow \text{Eve}^*(1^n), \quad k \leftarrow \{0, 1\}^n, \quad b \leftarrow \{0, 1\}, \quad i_1, \dots, i_t \leftarrow [1, 2^n], \quad \forall j \in [1, t] : c_j := m_{b,j} \oplus F_k(i_j).$$

Building a PRF distinguisher D that uses Eve^* as a subroutine. We construct an oracle distinguisher $D^{O(\cdot)}(1^n)$ for the PRF F , whose job is to tell apart the two worlds of

Definition 1: $O(\cdot) = F_k(\cdot)$ for a random k , versus $O(\cdot) = R(\cdot)$ for a truly random function $R \leftarrow \mathcal{F}_n$. The construction of D closely mirrors the distinguisher built in Lecture 07, Theorem 1, for the computational-OTP setting.

Pseudocode for $D^{O(\cdot)}(1^n)$:

1. Call the subroutine $\text{Eve}^*(1^n)$ and obtain its two challenge message sequences $(\bar{m}_0, \bar{m}_1) = ((m_{0,1}, \dots, m_{0,t}), (m_{1,1}, \dots, m_{1,t}))$.
2. Toss a coin $b \leftarrow \{0, 1\}$.
3. Sample t random indices $i_1, \dots, i_t \leftarrow [1, 2^n]$.
4. For each $j \in [1, t]$:
 - (a) Query the oracle $O(i_j)$ to obtain y_{i_j} .
 - (b) Set $c_j := m_{b,j} \oplus y_{i_j}$.
5. Call the subroutine $\text{Eve}^*(c_1, \dots, c_t)$ (recall Eve^* is *stateful*, so it remembers $\bar{m}_0, \bar{m}_1$ from step 1) to obtain its guess b' .
6. If $b' = b$, output 0; else output 1.

Note on efficiency. D runs Eve^* (a PPT algorithm) as a black-box subroutine exactly twice, samples $t = \text{poly}(n)$ random indices, and makes t oracle queries – so D itself is a PPT oracle algorithm, as required.

6.3 Analysis: what D 's two worlds correspond to

Proof

We analyze D 's behaviour in each of its two possible worlds, exactly mirroring the analysis of Theorem 1 from Lecture 07.

World 1: $O(\cdot) = F_k(\cdot)$ for $k \leftarrow \{0, 1\}^n$. In this world, D 's simulation of the challenger is *perfect*: the ciphertexts $c_j = m_{b,j} \oplus F_k(i_j)$ that D feeds to Eve^* are *exactly* distributed as real IND⁺-EAV-game ciphertexts for Construction 1. Hence, by our contradiction hypothesis (I),

$$\Pr \left[\text{Eve}^*(c_1, \dots, c_t) = b \right] - \frac{1}{2} \geq \frac{1}{p(n)} \implies \Pr_{k \leftarrow \{0, 1\}^n} \left[D^{F_k(\cdot)}(1^n) = 0 \right] = \Pr[\text{Eve}^*(\dots) = b] \geq \frac{1}{2} + \frac{1}{p(n)}. \quad (\text{I}')$$

(Recall D outputs 0 exactly when $b' = b$, i.e. exactly when Eve^* “wins”.)

World 2: $O(\cdot) = R(\cdot)$ for $R \leftarrow \mathcal{F}_n$. Here, every ciphertext $c_j = m_{b,j} \oplus R(i_j)$ is being masked by a value $R(i_j)$ where R is a *genuinely uniformly random function*. Since the indices $i_1, \dots, i_t$ are sampled uniformly and independently from an exponentially large set $[1, 2^n]$, they are *pairwise distinct* except with negligible probability (this is exactly the birthday-bound argument from Section 3.1, now with an exponential-size index space, so collisions are negligible). Conditioned on all indices being distinct, $R(i_1), \dots, R(i_t)$ are *t independent, uniformly random n -bit strings* (this is precisely what it means for R to be a uniformly random function: its values at distinct points are independent and uniform). Hence **each message $m_{b,j}$ is being masked by a genuine, fresh one-time pad** – exactly as in a perfectly-secret OTP encryption. By the perfect secrecy of the OTP, the ciphertexts $(c_1, \dots, c_t)$ carry *zero information* about b , so no algorithm, however powerful, can guess b with probability better than $\frac{1}{2}$:

$$\Pr_{R \leftarrow \mathcal{F}_n} \left[D^{R(\cdot)}(1^n) = 0 \right] = \Pr[\text{Eve}^*(\dots) = b] = \frac{1}{2}. \quad (\text{II})$$

6.4 Putting it together: contradicting the pseudorandomness of F

Proof

Combining (I') and (II), for infinitely many n :

$$\left| \Pr_{k \leftarrow \{0,1\}^n} [D^{F_k(\cdot)}(1^n) = 0] - \Pr_{R \leftarrow \mathcal{F}_n} [D^{R(\cdot)}(1^n) = 0] \right| = \left| \left(\frac{1}{2} + \frac{1}{p(n)} \right) - \frac{1}{2} \right| = \frac{1}{p(n)}.$$

Since $1/p(n)$ is *non-negligible* (as p is a fixed polynomial), this directly **contradicts the assumption that F is a PRF** (Definition 1 requires this quantity to be negligible for *every* PPT distinguisher, and we've just exhibited a PPT distinguisher D for which it isn't). **Contradiction!**

Conclusion. No such PPT Eve^* can exist. Hence Construction 1 is multi-message computationally secret against eavesdroppers, whenever F is a PRF. ■

[ADDED BY CLAUDE] A one-paragraph summary of the proof idea, in plain English

The whole proof is a single “swap the PRF for a truly random function, and see what changes” trick. If F really is pseudorandom, no efficient process (including one that has Eve^* embedded inside it) can tell whether it's interacting with F_k or a genuinely random function R . But if it *were* interacting with a genuinely random R , then (as long as the random indices i_j don't collide, which happens except with negligible probability) every message is masked by an honest, freshly-sampled, never-reused one-time pad – and one-time pads are *perfectly* secret, so Eve^* 's success probability would be forced down to exactly $\frac{1}{2}$ in that imaginary world. If Eve^* actually does better than $\frac{1}{2}$ (by a non-negligible amount) in the *real* world (using F_k), that gap *is itself* a way to distinguish F_k from R – contradicting the assumption that F is a PRF. This “replace real primitive by an ideal one, use the adversary's success gap as your distinguishing advantage” pattern is one of the most common proof techniques in all of cryptography, and you will see it again and again throughout the course (it is the same basic pattern used for the PRG-based hybrid argument in Lecture 08, just with a PRF in place of a PRG, and the OTP argument in place of the length-extension argument).

7 Application: Watermarking Large Language Models

The slides close with a modern application of PRFs: statistically watermarking text generated by a language model like Claude, so that the model’s operator can later test, with high confidence, whether a given piece of text was produced by their model.

7.1 Goal

Anthropic wants to *privately* test whether a given piece of text was generated using Claude. “Privately” here means: the test should use a secret key that only Anthropic knows, so that an outside party cannot easily figure out the watermarking rule and remove or forge it.

7.2 How watermarking tokens works, at inference time

- Ordinarily, to generate its next token, a language model samples randomly from a probability distribution over a table of candidate next tokens (e.g. *important*: 40%, *significant*: 30%, *substantial*: 20%, *notable*: 10%, as illustrated on the slide).
- With watermarking turned on, a **(bit-valued) PRF**, keyed with a secret key known only to Anthropic, is applied to the recent context (the sequence of prior tokens) to *tag* each candidate token in the table as **green** or **red**. Since the tag is produced by a PRF (not a literal random coin), it is *deterministic given the context and the key*, so it can be recomputed later, but it is *unpredictable* to anyone who does not know the key.
- The sampling distribution is then *nudged*: the probability mass on green-tagged tokens is increased slightly, and the probability mass on red-tagged tokens is decreased correspondingly (as shown in the slide’s before/after bar-chart example: “important” and “substantial” get boosted from 40%/20% to $\approx 53\%$ / $\approx 28\%$, while “significant”/“notable” shrink).

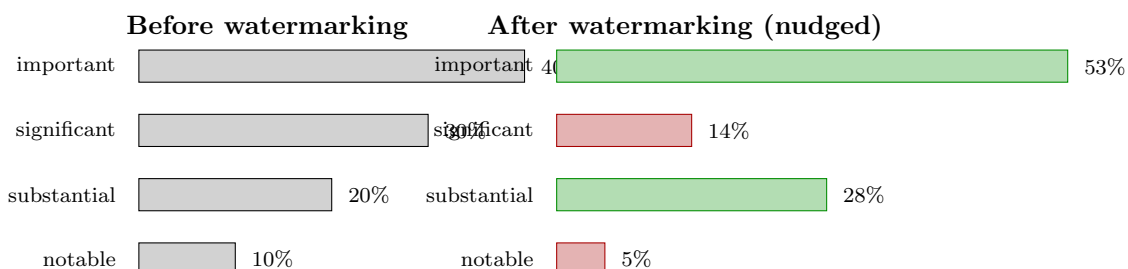


Figure 5: Redrawn from the slide (originally credited to deClaude.org): before watermarking (left), the model’s raw next-token distribution; after watermarking (right), tokens that the secret PRF tags **green** (e.g. “important”, “substantial”) get a probability boost, while tokens tagged **red** (e.g. “significant”, “notable”) get suppressed.

7.3 Detecting the watermark

Given a suspect piece of text, Anthropic (who knows the secret PRF key) recomputes, for every token in the text, whether the PRF would have tagged it green or red *given its preceding context*. In genuinely Claude-generated (watermarked) text, green tokens will appear *more often* than they would by chance, because the sampling process was nudged towards them at generation time. In text that was *not* generated by Claude (e.g. human-written text, or text from an unrelated model), the green/red tags – computed using Anthropic’s secret PRF – are essentially just a random labelling uncorrelated with which words the author actually chose, so green and red tokens should appear roughly equally often. A statistical test (e.g. counting the

fraction of green tokens and comparing it against the null hypothesis of $\approx 50\%$) then yields a confidence score for “this text was/was not produced by Claude.”

[ADDED BY CLAUDE] Why must the watermarking function be a *PRF* specifically?

- **Reproducibility (needs to be a deterministic function of the key and context):** the green/red tag for a given context must be *recomputable* later, during detection, using only the secret key – so it cannot be a fresh random coin flip at generation time; it must be some *function* of (key, context).
- **Unpredictability/unforgeability (needs to be pseudorandom):** if the tagging rule were predictable or easily learned by an outsider without the key (e.g. if it were just “tag word w green if w ’s first letter is in A–M”), then anyone could reverse-engineer which words are currently “green” and deliberately write human text that is majority-green, *forging* a positive watermark detection, or conversely, an adversary could paraphrase Claude’s output by swapping green tokens for synonymous red tokens, *erasing* the real watermark. Because the tagging function is a PRF keyed by a secret only Anthropic knows, no efficient outsider (without the key) can predict tags better than random guessing – exactly the PRF security guarantee of Definition 1, with the context playing the role of the query input x .
- **Exactly matches the PRF interface:** the tagging function needs to be efficiently evaluated on an exponentially large space of possible contexts (all possible token histories), without ever precomputing a giant lookup table – precisely the “exponential domain, efficient local access” property that motivated PRFs over plain PRGs in Section 3.

This is a nice illustration of how the *exact same abstract primitive* (a PRF satisfying Definition 1) shows up in two superficially very different applications: encrypting many messages (Construction 1) and statistically fingerprinting model outputs (LLM watermarking) – underscoring why cryptographers care about nailing down clean, reusable definitions like Definition 1.

The actual watermarking scheme referenced was proposed by Kirchenbauer et al. [4] (ICML 2023); both Google’s SynthID and Anthropic’s own watermarking approach build on ideas from that line of work.

8 Recap and What's Next

- We introduced **Pseudorandom Functions (PRFs)** as the computational (efficiently realizable) analogue of random oracles.
- PRF security is defined via **query (oracle) access**, not by handing over a full description of the function – this is what lets the definition make sense for a PPT distinguisher.
- We constructed a **stateless multi-message SKE** from any PRF (Construction 1), proved it secure (Theorem 2) via a reduction to the PRF's pseudorandomness combined with the perfect secrecy of the one-time pad, and saw that the reduction requires an *exponentially large* index space to keep birthday collisions negligible.
- **Applications of PRFs** mentioned:
 - Watermarking LLM outputs (Section 6, in detail).
 - Message authentication (previewed for a future lecture, around Lecture 12).
 - *Natural proofs*: PRFs also show up as a formal *barrier* to certain kinds of attacks on the $P \stackrel{?}{=} NP$ question – if sufficiently strong PRFs exist, this rules out a broad class of “natural” proof techniques from resolving $P \neq NP$ (see Razborov–Rudich; further reading in [1] §4 or [2]).
- **Next lecture:** *How* do we actually *construct* a PRF from more basic assumptions? (Definition 1 only tells us what a PRF must satisfy, not how to build one!) We will also move to a **stronger attack model**: chosen-plaintext attacks (CPA), where the adversary can ask the challenger to encrypt messages of its choosing *before* the challenge phase, not just passively eavesdrop.

[ADDED BY CLAUDE] A preview, for motivation, of the next lecture's punchline

PRFs are constructed from PRGs via the celebrated **Goldreich–Goldwasser–Micali (GGM) construction** [3]: given a length-doubling PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, split $G(s)$ into two n -bit halves $G_0(s), G_1(s)$, and define a binary tree of depth n where you walk down the tree following the bits of the input $x = x_1x_2 \cdots x_n$, applying G_{x_1} , then G_{x_2} to the result, and so on. The value at the leaf reached after following all n bits of x is defined to be $F_k(x)$. This is a beautiful illustration of exactly the “exponential domain, efficient local access” idea from Section 3: the conceptual full tree has 2^n leaves (exponentially many), but computing any single leaf's value only requires n evaluations of G (polynomial time), because you only ever walk one root-to-leaf path.

[ADDED BY CLAUDE] ADDED: A worked toy example to build intuition for the GGM tree

Since the GGM construction can feel abstract on first sight, here is a fully worked-out toy example with $n = 2$ -bit inputs, so the tree has depth 2 and $2^2 = 4$ leaves. Let $G : \{0, 1\}^2 \rightarrow \{0, 1\}^4$ be a length-doubling PRG (toy-sized only for illustration; a real PRG needs a much larger security parameter). Write $G(s) = G_0(s) \| G_1(s)$ where $G_0(s), G_1(s) \in \{0, 1\}^2$ are the left and right halves.

Key. Sample $k \leftarrow \{0, 1\}^2$, say $k = 01$.

Evaluating $F_k(x)$ for $x = x_1x_2$:

- *Level 1 (root $\rightarrow$ depth 1):* compute $G(k)$, and take $G_{x_1}(k)$ – i.e. the left half if

$x_1 = 0$, the right half if $x_1 = 1$. Call this value $v_1 \in \{0, 1\}^2$.

- *Level 2 (depth 1 $\rightarrow$ leaf)*: compute $G(v_1)$, and take $G_{x_2}(v_1)$. This 2-bit string is $F_k(x)$.

So, e.g., $F_k(00) = G_0(G_0(k))$, $F_k(01) = G_1(G_0(k))$, $F_k(10) = G_0(G_1(k))$, and $F_k(11) = G_1(G_1(k))$ – four different leaves, each reached by a distinct root-to-leaf path, each computed with only *two* calls to G . Notice that if n were, say, 256 (a realistic security parameter) there would be 2^{256} conceptual leaves, but every single evaluation $F_k(x)$ would still cost only 256 invocations of G – this is exactly the “exponential domain, polynomial-time local access” property PRFs are built to have.

Sources and Further Reading

References

- [1] Scott Aaronson. *Is P versus NP formally independent?* Bull. EATCS, 81:109–136, 2003.
- [2] Timothy Y. Chow. *What is... a natural proof?* Notices of the AMS, 58(11):1586–1587, 2011.
- [3] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. *How to construct random functions (extended abstract)*. In 25th FOCS, pages 464–479. IEEE Computer Society Press, October 1984.
- [4] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. *A watermark for large language models*. In ICML, volume 202 of Proceedings of Machine Learning Research, pages 17061–17084. PMLR, 2023.
- [5] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography* (3rd ed.). Chapman and Hall/CRC, 2014. [See §3.5 for a formal description of a construction analogous to Construction 1.]